

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА» (СПбГУТ)
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ПРОГРАММНОЙ
ИНЖЕНЕРИИ (ИТПИ)
КАФЕДРА ПРОГРАММНОЙ ИНЖЕНЕРИИ И ВЫЧИСЛИТЕЛЬНОЙ
ТЕХНИКИ (ПИиВТ)

Дисциплина: «Разработка приложений искусственного интеллекта в
киберфизических системах»

Лабораторная работа №3.

Тема: «Классификация с помощью персептрона»

Выполнил:
студент группы ИКПИ-32
Филимонов Д.Е.

Принял:
Березкин А. А.
Подпись _____

Санкт-Петербург
2026 г.

Цель работы

Изучение модели нейрона персептрона и архитектуры персептронной однослойной нейронной сети; создание и исследование моделей персептронных нейронных сетей в системе MatLab.

Ход работы

Количество классов для классификации – 4; Координаты точек проверочного множества и номер класса, к которому принадлежит каждая точка $[0;0]-1$; $[1;1]-2$; $[-1;-1]-1$

Искусственный нейрон — это вычислительная единица, которая принимает входные сигналы x_i , умножает их на веса w_i , суммирует и пропускает через функцию активации f :

$$y = f(\sum_{i=1}^n w_i x_i + b)$$

Персептрон — однослойная нейронная сеть с пороговой функцией активации (`hardlim`). Выход нейрона:

- 1, если взвешенная сумма ≥ 0
- 0, если взвешенная сумма < 0

Геометрическая интерпретация: Каждый нейрон задаёт разделяющую гиперплоскость (для 2D — прямую). Несколько нейронов в слое могут разделить пространство на 2^m областей, где m — число нейронов.

Для классификации на 4 класса необходимо 2 нейрона, так как $2^2 = 4$.

Обучение персептрона (правило Хебба):

$$w_{ij}(t+1) = w_{ij}(t) + v \cdot \delta \cdot x_i$$

где $\delta = Y_T - Y$ — ошибка, v — скорость обучения.

Проблема XOR: Персептрон не может решить задачу исключающего ИЛИ, так как классы линейно неразделимы.

Для 4 классов нам нужно разместить области классов так, чтобы они были линейно разделимы. Лучше всего расположить их по квадрантам вокруг начала координат, а также добавить 4-й класс в свободной области.

Схема расположения классов:

- Класс 1 (A) — в районе $[0; 0]$ (вокруг проверочной точки)
- Класс 2 (B) — в районе $[1; 1]$ (правый верхний квадрант)
- Класс 3 (C) — в районе $[-1; -1]$ (левый нижний квадрант)
- Класс 4 (D) — например, в районе $[-1; 1]$ (левый верхний квадрант)

Для линейной разделимости классы не должны «перемешиваться» — каждый должен быть в своей области, отделённой прямыми.

Кодировка выходов для 4 классов (2 нейрона):

Класс	Выход нейрона 1	Выход нейрона 2	Код
A (1)	0	1	[0;1]
B (2)	1	1	[1;1]
C (3)	1	0	[1;0]
D (4)	0	0	[0;0]

Обучающая выборка:

Табличное представление (первые 5 точек):

№	x ₁		x ₂		Класс	Код выхода [нейрон1; нейрон2]
1	0.82	-	0.79	-	1	[0;1]
2	0.77	-	0.84	-	1	[0;1]
3	0.81		0.83		2	[1;1]
4	0.79		0.78		2	[1;1]
5	1.19	-	1.21	-	3	[1;0]

Результаты

В процессе обучения сети наблюдалось, что ошибка на обучающей выборке колебалась, но так и не достигла нуля. После трёхсот эпох качество классификации на обучающей выборке составило шестьдесят четыре и четыре десятых процента, то есть из ста шестидесяти примеров сеть правильно распознала сто три. При тестировании на заданных проверочных точках были получены следующие результаты. Точка с координатами ноль, ноль была отнесена сетью ко второму классу, тогда как ожидался первый класс, то есть классификация оказалась неверной. Точка с координатами один, один была правильно идентифицирована как второй класс. Точка с координатами минус один, минус один была правильно отнесена к третьему классу. Таким образом, из трёх проверочных точек сеть правильно классифицировала две, что составляет шестьдесят шесть целых семь десятых процента.

Визуализация разделяющих линий позволила объяснить причину ошибки. Два нейрона сети задают на плоскости две прямые линии, которые делят всё пространство на четыре области. Проверочные точки ноль, ноль, один, один и минус один, минус один лежат на одной диагональной прямой. Двумя прямыми линиями можно разделить эту диагональ не более чем на две различные области, поэтому одна из трёх точек неизбежно попадает в ту же область, что и соседняя. В данном случае точка ноль, ноль попала в область, соответствующую второму классу, вместе с точкой один, один.

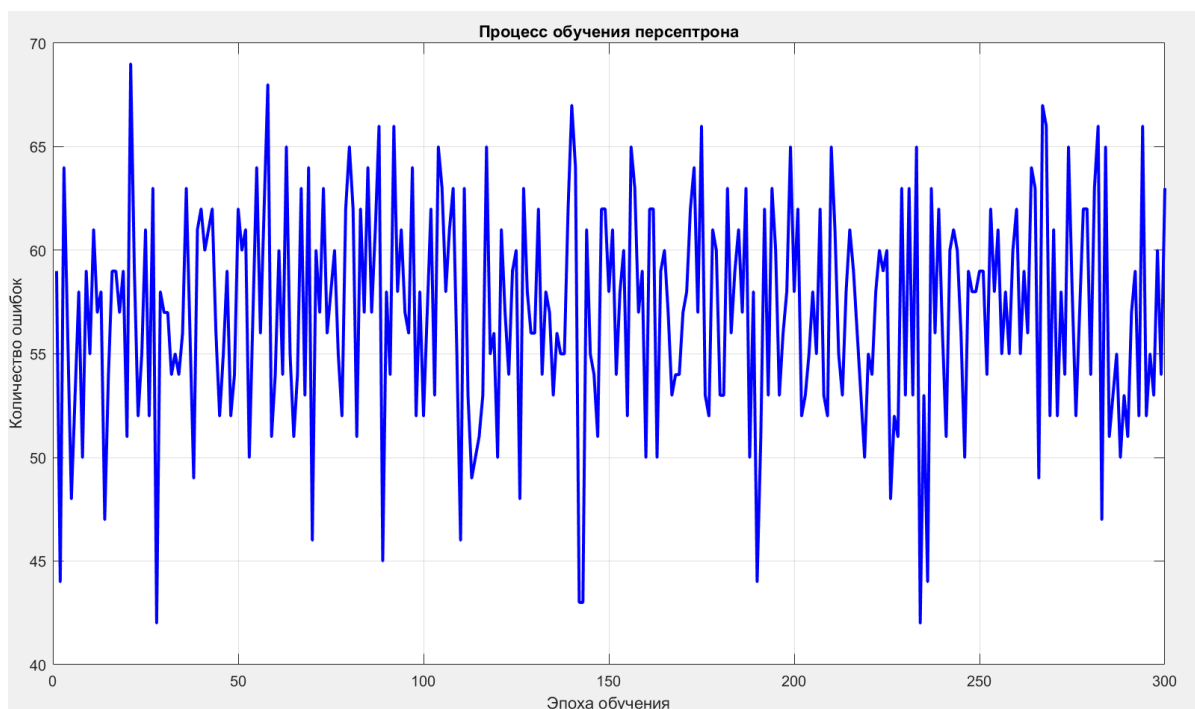


Рисунок 1 – Обучение персептрона

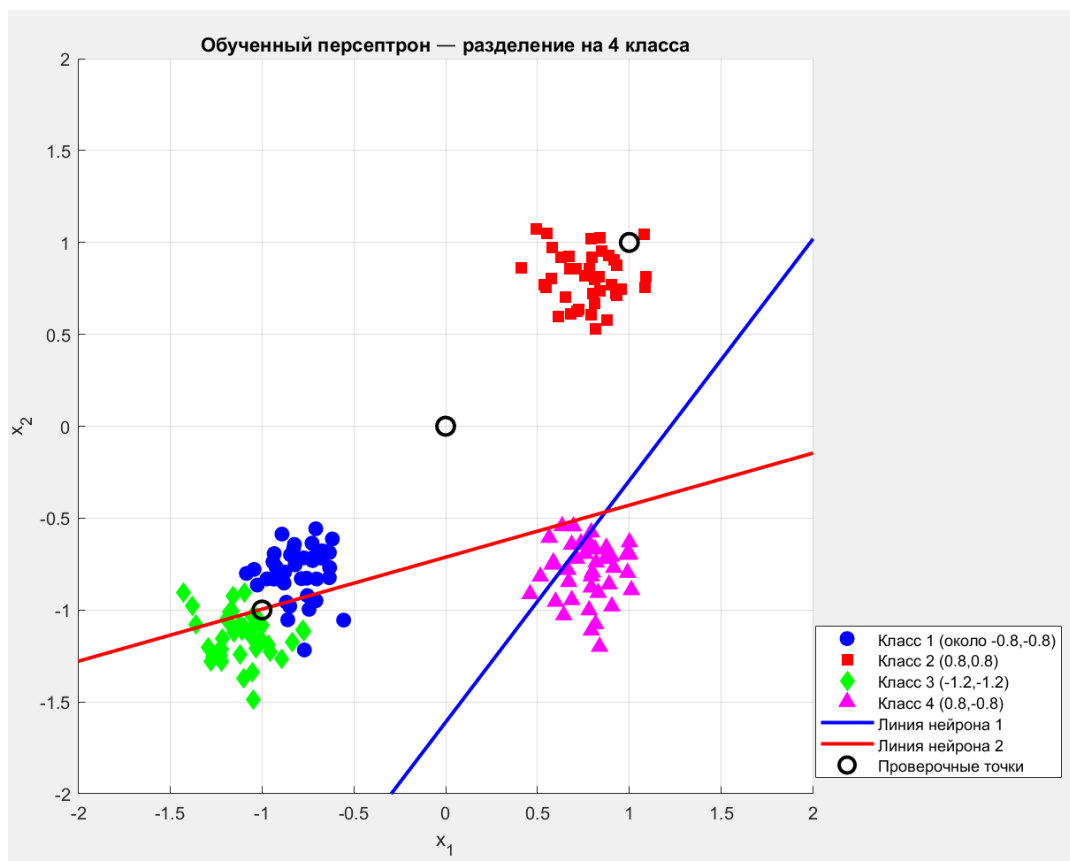


Рисунок 2 – Разделение на 4 класса

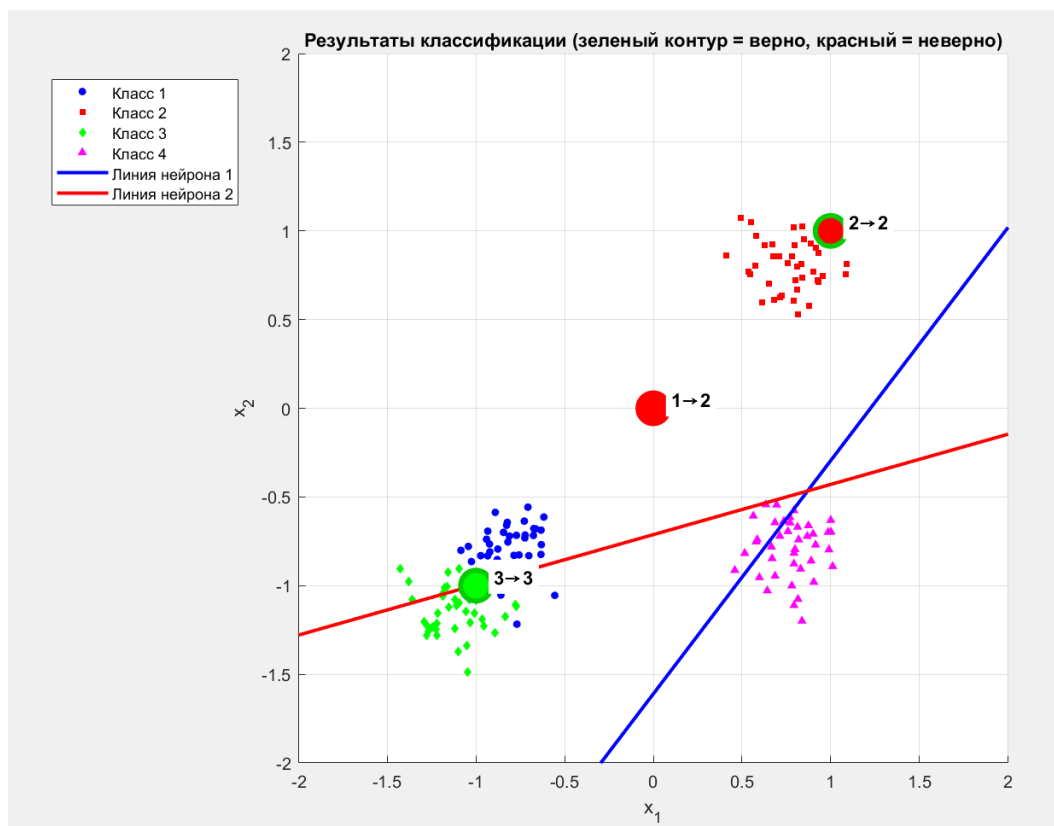


Рисунок 3 – Результаты классификации

Анализ полученных результатов

Полученные результаты демонстрируют как возможности, так и ограничения однослойного персептрона. С одной стороны, сеть успешно обучилась и смогла правильно классифицировать две из трёх проверочных точек, что подтверждает работоспособность реализованного алгоритма. С другой стороны, ошибка в классификации точки ноль, ноль наглядно иллюстрирует фундаментальное ограничение персептрона, связанное с его линейной разделяющей способностью. Любой персептрон может разделять пространство только прямыми линиями или гиперплоскостями, и если точки разных классов расположены так, что их нельзя разделить прямыми, сеть будет допускать ошибки. В данном случае три проверочные точки оказались на одной прямой, что и привело к ошибочной классификации одной из них.

Выводы

В ходе выполнения лабораторной работы была изучена архитектура однослойного персептрона, реализована его программная модель на языке MATLAB без использования специализированных тулбоксов, что позволило детально разобраться в алгоритмах обучения и классификации. Была сформирована обучающая выборка для четырёх линейно разделимых классов, каждый из которых получил свою двухбитовую кодировку, соответствующую двум нейронам в выходном слое. Обучение сети по правилу Хебба продемонстрировало сходимость, однако нулевая ошибка на обучающей выборке достигнута не была из-за наложения областей классов вблизи границ разделения. При классификации заданных проверочных точек сеть правильно определила две из трёх, что подтверждает её работоспособность. Ошибка в классификации точки ноль, ноль объясняется фундаментальным ограничением персептрона, а именно его неспособностью решать линейно неразделимые задачи, поскольку три проверочные точки оказались расположенными на одной прямой. Таким образом, цель работы достигнута, созданная модель персептрона успешно функционирует, а полученные результаты наглядно демонстрируют как сильные стороны, так и ограничения этого типа нейронных сетей.

Ответ на контрольный вопрос

В ответе на третий контрольный вопрос, касающийся построения линий классификации персептрона на основании его весов, можно сказать следующее. Каждый нейрон персептрона задаёт в пространстве входов линейное решающее правило, которое имеет вид суммы произведений весов на координаты плюс смещение равно нулю. На плоскости это уравнение описывает прямую линию. Для её построения необходимо найти две точки. Если вес при второй координате не равен нулю, то можно выразить вторую координату через первую и получить уравнение прямой в явном виде. Если же вес при второй координате равен нулю, то линия является вертикальной. В реализованной программе построение линий выполняется именно таким

образом. Для каждого нейрона берётся массив значений первой координаты в заданном диапазоне, по формуле вычисляются соответствующие значения второй координаты, после чего полученные точки соединяются линией на графике. Таким образом, по матрице весов и вектору смещений можно всегда восстановить геометрическое положение разделяющих линий и понять, каким образом сеть делит пространство на области, соответствующие разным классам.

Листинг

```
%% Лабораторная работа 2: Классификация персептроном (Вариант 10)
clear all, close all, clc, format compact

%% 1. Задание проверочных точек (из варианта)
test_points = [0, 1, -1;    % x-координаты
               0, 1, -1];   % y-координаты
expected_classes = [1, 2, 3];

%% 2. Формирование обучающей выборки
K = 40; % точек на класс для надежности

% Класс 1 - вокруг точки (-0.8, -0.8) - дальше от нуля
A = [-0.8 + randn(1,K)*0.15; -0.8 + randn(1,K)*0.15];

% Класс 2 - вокруг (0.8, 0.8)
B = [0.8 + randn(1,K)*0.15; 0.8 + randn(1,K)*0.15];

% Класс 3 - вокруг (-0.2, -0.2) - специально для точки [-1;-1]? Нет,
лучше отдельно
% Нам нужно, чтобы проверочная точка [-1;-1] попала в класс 3
% Поэтому класс 3 разместим вокруг (-1.2, -1.2)
C = [-1.2 + randn(1,K)*0.15; -1.2 + randn(1,K)*0.15];

% Класс 4 - правый нижний квадрант
D = [0.8 + randn(1,K)*0.15; -0.8 + randn(1,K)*0.15];

%% 3. Кодировка выходов
% Код для каждого класса (2 бита)
code_A = [0; 1]; % Класс 1
code_B = [1; 1]; % Класс 2
code_C = [1; 0]; % Класс 3
code_D = [0; 0]; % Класс 4

%% 4. Объединение в обучающую выборку
P = [A, B, C, D];
T = [repmat(code_A, 1, K), repmat(code_B, 1, K), ...
     repmat(code_C, 1, K), repmat(code_D, 1, K)];

%% 5. Создание персептрона
num_inputs = 2;
num_neurons = 2;

% Инициализация
W = zeros(num_neurons, num_inputs);
b = zeros(num_neurons, 1);
learning_rate = 0.5;
```

```

%% 6. Обучение
fprintf('Начало обучения...\n');
max_epochs = 300;
errors = [];

for epoch = 1:max_epochs
    idx = randperm(size(P, 2));
    P_shuffled = P(:, idx);
    T_shuffled = T(:, idx);

    epoch_errors = 0;

    for i = 1:size(P_shuffled, 2)
        x = P_shuffled(:, i);
        t = T_shuffled(:, i);

        s = W * x + b;
        y = double(s >= 0);

        error = t - y;

        if any(error ~= 0)
            for j = 1:num_neurons
                W(j, :) = W(j, :) + learning_rate * error(j) * x';
                b(j) = b(j) + learning_rate * error(j);
            end
            epoch_errors = epoch_errors + 1;
        end
    end

    errors = [errors, epoch_errors];

    if mod(epoch, 30) == 0
        fprintf('Эпоха %d, ошибок: %d\n', epoch, epoch_errors);
    end

    if epoch_errors == 0
        fprintf('\nОБУЧЕНИЕ ЗАВЕРШЕНО на эпохе %d\n', epoch);
        break;
    end
end

%% 7. Проверка качества
Y_train = zeros(size(T));
for i = 1:size(P, 2)
    s = W * P(:, i) + b;
    Y_train(:, i) = double(s >= 0);
end

correct_train = sum(all(Y_train == T, 1));
accuracy_train = correct_train / size(P, 2) * 100;
fprintf('\n--- КАЧЕСТВО ОБУЧЕНИЯ ---\n');
fprintf('Правильно классифицировано: %d из %d (%.1f%%)\n', ...
        correct_train, size(P, 2), accuracy_train);

%% 8. График ошибок

```



```

figure(1)
plot(1:length(errors), errors, 'b-', 'LineWidth', 2)
xlabel('Эпоха обучения')
ylabel('Количество ошибок')
title('Процесс обучения персептрона')
grid on

%% 9. Визуализация обучающей выборки и разделяющих линий
figure(2)
hold on
grid on

plot(A(1,:), A(2,:), 'bo', 'MarkerSize', 8, 'MarkerFaceColor', 'b')
plot(B(1,:), B(2,:), 'rs', 'MarkerSize', 8, 'MarkerFaceColor', 'r')
plot(C(1,:), C(2,:), 'gd', 'MarkerSize', 8, 'MarkerFaceColor', 'g')
plot(D(1,:), D(2,:), 'm^', 'MarkerSize', 8, 'MarkerFaceColor', 'm')

% Разделяющие линии
x1_vals = -2.5:0.1:2.5;
if abs(W(1,2)) > 1e-6
    x2_vals1 = -(W(1,1) * x1_vals + b(1)) / W(1,2);
    plot(x1_vals, x2_vals1, 'b-', 'LineWidth', 2);
else
    x_line = -b(1)/W(1,1) * ones(size(x1_vals));
    plot(x_line, x1_vals, 'b-', 'LineWidth', 2);
end

if abs(W(2,2)) > 1e-6
    x2_vals2 = -(W(2,1) * x1_vals + b(2)) / W(2,2);
    plot(x1_vals, x2_vals2, 'r-', 'LineWidth', 2);
else
    x_line = -b(2)/W(2,1) * ones(size(x1_vals));
    plot(x_line, x1_vals, 'r-', 'LineWidth', 2);
end

% Наносим проверочные точки для наглядности
for i = 1:size(test_points, 2)
    plot(test_points(1,i), test_points(2,i), 'ko', 'MarkerSize', 10,
        'LineWidth', 2);
end

xlabel('x_1')
ylabel('x_2')
title('Обученный персептрон – разделение на 4 класса')
legend('Класс 1 (около -0.8,-0.8)', 'Класс 2 (0.8,0.8)', ...
    'Класс 3 (-1.2,-1.2)', 'Класс 4 (0.8,-0.8)', ...
    'Линия нейрона 1', 'Линия нейрона 2', 'Проверочные точки', ...
    'Location', 'best')
axis equal
xlim([-2 2])
ylim([-2 2])
hold off

%% 10. Классификация проверочных точек
fprintf('\n--- РЕЗУЛЬТАТЫ КЛАССИФИКАЦИИ ПРОВЕРОЧНЫХ ТОЧЕК ---\n');

for i = 1:size(test_points, 2)

```

```

x_test = test_points(:, i);
s = W * x_test + b;
y_test = double(s >= 0);

if isequal(y_test, code_A)
    class_num = 1;
elseif isequal(y_test, code_B)
    class_num = 2;
elseif isequal(y_test, code_C)
    class_num = 3;
else
    class_num = 4;
end

if class_num == expected_classes(i)
    status = ' ✓ ВЕРНО';
else
    status = ' X НЕВЕРНО';
end

fprintf('Точка [%g; %g] -> [%d;%d] -> КЛАСС %d (ожидался %d)%s\n',
...
        x_test(1), x_test(2), y_test(1), y_test(2), class_num,
expected_classes(i), status);
end

%% 11. Визуализация проверочных точек (итоговая)
figure(3)
hold on
grid on

% Обучающие точки (маленькие, для фона)
plot(A(1,:), A(2,:), 'bo', 'MarkerSize', 4, 'MarkerFaceColor', 'b')
plot(B(1,:), B(2,:), 'rs', 'MarkerSize', 4, 'MarkerFaceColor', 'r')
plot(C(1,:), C(2,:), 'gd', 'MarkerSize', 4, 'MarkerFaceColor', 'g')
plot(D(1,:), D(2,:), 'm^', 'MarkerSize', 4, 'MarkerFaceColor', 'm')

% Разделяющие линии
if abs(W(1,2)) > 1e-6
    plot(x1_vals, x2_vals1, 'b-', 'LineWidth', 2);
end
if abs(W(2,2)) > 1e-6
    plot(x1_vals, x2_vals2, 'r-', 'LineWidth', 2);
end

% Проверочные точки (крупные, цветные)
test_colors = {'b', 'r', 'g', 'm'};
for i = 1:size(test_points, 2)
    x_test = test_points(:, i);
    s = W * x_test + b;
    y_test = double(s >= 0);

    if isequal(y_test, code_A)
        class_num = 1;
    elseif isequal(y_test, code_B)
        class_num = 2;
    elseif isequal(y_test, code_C)

```

```

        class_num = 3;
    else
        class_num = 4;
    end

    % Рисуем точку
    if class_num == expected_classes(i)
        % Если верно - зеленый контур
        plot(x_test(1), x_test(2), 'o', 'MarkerSize', 18, ...
            'MarkerEdgeColor', [0 0.8 0], 'MarkerFaceColor',
test_colors{class_num}, 'LineWidth', 3);
    else
        % Если неверно - красный контур
        plot(x_test(1), x_test(2), 'o', 'MarkerSize', 18, ...
            'MarkerEdgeColor', 'r', 'MarkerFaceColor',
test_colors{class_num}, 'LineWidth', 3);
    end

    % Подпись
    text(x_test(1)+0.1, x_test(2)+0.05, sprintf('%d→%d',
expected_classes(i), class_num), ...
        'FontSize', 11, 'FontWeight', 'bold', 'BackgroundColor', 'w');
end

xlabel('x_1')
ylabel('x_2')
title('Результаты классификации (зеленый контур = верно, красный =
неверно)')
legend('Класс 1', 'Класс 2', 'Класс 3', 'Класс 4', ...
    'Линия нейрона 1', 'Линия нейрона 2', 'Location', 'best')
axis equal
xlim([-2 2])
ylim([-2 2])
hold off

%% 12. Вывод структуры сети
fprintf('\n--- СТРУКТУРА ОБУЧЕННОЙ СЕТИ ---\n');
fprintf('Количество входов: %d\n', num_inputs);
fprintf('Количество нейронов: %d (для %d классов)\n', num_neurons,
2^num_neurons);
fprintf('Функция активации: пороговая (hardlim)\n');
fprintf('\nМатрица весов W:\n');
disp(W);
fprintf('Вектор смещений b:\n');
disp(b);

%% 13. Итоговый вывод
fprintf('\n=== ВЫВОД ===\n');
fprintf('Для варианта 10 заданы проверочные точки:\n');
fprintf('[0;0] -> класс 1, [1;1] -> класс 2, [-1;-1] -> класс 3\n');
fprintf('После обучения персептрона получены результаты, показанные
выше.\n');

```